Container Monitoring with Wazuh SIEM

1. Introduction

Docker is one of the most widely used container platforms in modern IT infrastructure. It allows applications to run in isolated environments called containers. Because containers are lightweight and easy to deploy, they are used heavily in development, testing, and production environments.

However, containers also introduce new security risks. A compromised container can affect the host system or other containers running alongside it. This is why monitoring Docker container activity is an essential part of any security strategy.

Wazuh provides a built-in Docker listener module that connects directly to the Docker daemon on a monitored machine. Once enabled, it captures real-time events such as container start, stop, pause, kill, and image pull operations, and forwards them to the Wazuh Manager for analysis and alerting.

In this use case, we install Docker Engine on the Ubuntu Server (where the Wazuh agent is already installed), configure the Wazuh agent to monitor Docker events, and verify that the alerts appear correctly on the Wazuh Dashboard.

2. Lab Environment

Component	Details
Wazuh Manager	Installed on Ubuntu Desktop (Oracle VirtualBox) acts as the central server that receives and analyzes all security events.
Wazuh Agent	Installed on Ubuntu Server (Oracle VirtualBox) where Docker Engine is installed and monitored.

Communication	Agent forwards Docker events to Manager over the network in real-time.
Goal	Monitor Docker container lifecycle events (start, stop, kill, image pull, etc.) and generate alerts on the Wazuh Dashboard.

3. What We Are Doing

We are setting up Docker container monitoring in three main phases:

- Install Docker Engine on the Ubuntu Server where the Wazuh agent is already running.
- Configure the Wazuh agent to enable its built-in Docker listener module so it can capture container events.
- Test the monitoring by starting, stopping, and interacting with Docker containers and verifying that Wazuh generates the correct alerts.

STEP 1

Install Docker Engine on Ubuntu Server

Log in to your Ubuntu Server virtual machine. All the commands in this step are run on the Ubuntu Server, the machine where your Wazuh agent is installed.

1.1 Remove Any Old Docker Packages

First, remove any older or conflicting Docker packages that might already be on the system. This prevents version conflicts during installation.

sudo apt remove docker docker-engine docker.io containerd runc -y

```
ubuntu@ubuntu:~$ sudo apt remove docker docker-engine docker.io containerd runc -y
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package docker-engine
ubuntu@ubuntu:~$
```

1.2 Update the System Package List

Update the package index so apt knows about the latest available packages.

sudo apt update

```
ubuntu@ubuntu:~$ sudo apt update
Hit:1 http://pk.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 https://dl.google.com/linux/chrome-stable/deb stable InRelease [1,825 B]
Get:4 https://dl.google.com/linux/chrome-stable/deb stable/main amd64 Packages [1,213 B]
Get:5 http://pk.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [997 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [813 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,439 kB]
Get:10 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [114 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.6 kB]
Get:12 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [806 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,268 kB]
Get:14 http://pk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [360 kB]
Get:15 http://pk.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:16 http://pk.archive.ubuntu.com/ubuntu jammy-backports/main amd64 DEP-11 Metadata [7,368 B]
Get:17 http://pk.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 DEP-11 Metadata [12.4 kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,173 kB]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.6 kB]
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,029 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [694 kB]
Get:22 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 DEP-11 Metadata [125 kB]
Fetched 13.3 MB in 12s (1,123 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
N: Skipping acquisition of configured file 'main/binary-i386/Packages', as repository 'https://dl.google.com/linux/chrome-stable/deb
 stable InRelease' doesn't support architecture 'i386'
ubuntu@ubuntu:~$
```

1.3 Install Required Dependencies

Install the packages that are needed to allow apt to download packages over HTTPS.

sudo apt install -y ca-certificates curl gnupg lsb-release

```
ubuntu@ubuntu:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
lsb-release is already the newest version (11.1.0ubuntu4).
lsb-release set to manually installed.
ca-certificates is already the newest version (20240203~22.04.1).
ca-certificates set to manually installed.
curl is already the newest version (7.81.0-1ubuntu1.23).
gnupg is already the newest version (2.2.27-3ubuntu2.5).
gnupg set to manually installed.
0 to upgrade, 0 to newly install, 0 to remove and 40 not to upgrade.
ubuntu@ubuntu:~$
```

1.4 Add Docker's Official GPG Key

Docker packages are signed with a GPG key. Adding this key tells your system that the packages it downloads are genuine and have not been tampered with.

sudo mkdir -p /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor --yes -o /etc/apt/keyrings/docker.gpg

```
ubuntu@ubuntu:~$ sudo mkdir -p /etc/apt/keyrings
ubuntu@ubuntu:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor --yes -o /etc/apt/keyrings/docker.gpg
ubuntu@ubuntu:~$
```

1.5 Add the Docker Repository

Add the official Docker repository to your apt sources so you can install the latest Docker Engine.

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

```
ubuntu@ubuntu:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/li
nux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
ubuntu@ubuntu:~$
```

1.6 Install Docker Engine

Update the package list again (now including Docker's repository) and install Docker Engine along with the container runtime and Docker Compose plugin.

sudo apt update

```
ubuntu@ubuntu:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [75.6 kB]
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:5 http://pk.archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 http://pk.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 http://pk.archive.ubuntu.com/ubuntu jammy-backports InRelease
Fetched 124 kB in 1s (114 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
N: Skipping acquisition of configured file 'main/binary-i386/Packages', as repository 'https://dl.google.com/linux/chrome-stable/deb
 stable InRelease' doesn't support architecture 'i386'
```

sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```
ubuntu@ubuntu:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras libslirp0 pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin
The following NEW packages will be installed
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz
  slirp4netns
0 to upgrade, 9 to newly install, 0 to remove and 40 not to upgrade.
Need to get 94.9 MB of archives.
After this operation, 365 MB of additional disk space will be used.
Get:1 https://download.docker.com/linux/ubuntu jammy/stable amd64 containerd.io amd64 2.2.3-1~ubuntu.22.04~jammy [23.6 MB]
Get:2 http://pk.archive.ubuntu.com/ubuntu jammy/universe amd64 pigz amd64 2.6-1 [63.6 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu jammy/main amd64 libslirp0 amd64 4.6.1-1build1 [61.5 kB]
Get:4 http://pk.archive.ubuntu.com/ubuntu jammy/universe amd64 slirp4netns amd64 1.0.1-2 [28.2 kB]
Get:5 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-cli amd64 5:29.4.1-1~ubuntu.22.04~jammy [17.0 MB]
Get:6 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:29.4.1-1~ubuntu.22.04~jammy [22.7 MB]
Get:7 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-buildx-plugin amd64 0.33.0-1~ubuntu.22.04~jammy [16.9 MB]
Get:8 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-rootless-extras amd64 5:29.4.1-1~ubuntu.22.04~jammy [6,5
60 kB]
Get:9 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-compose-plugin amd64 5.1.3-1~ubuntu.22.04~jammy [8,023 kB]
Fetched 94.9 MB in 20s (4,637 kB/s)
Selecting previously unselected package containerd.io.
(Reading database ... 215813 files and directories currently installed.)
Preparing to unpack .../0-containerd.io_2.2.3-1~ubuntu.22.04~jammy_amd64.deb ...
Unpacking containerd.io (2.2.3-1~ubuntu.22.04~jammy) ...
Selecting previously unselected package docker-ce-cli.
Preparing to unpack .../1-docker-ce-cli_5%3a29.4.1-1~ubuntu.22.04~jammy_amd64.deb ...
Unpacking docker-ce-cli (5:29.4.1-1~ubuntu.22.04~jammy) ...
Selecting previously unselected package docker-ce.
Preparing to unpack .../2-docker-ce_5%3a29.4.1-1~ubuntu.22.04~jammy_amd64.deb ...
Unpacking docker-ce (5:29.4.1-1~ubuntu.22.04~jammy) ...
Selecting previously unselected package pigz.
Preparing to unpack .../3-pigz_2.6-1_amd64.deb ...
Unpacking pigz (2.6-1) ...
```

1.7 Verify Docker is Running

Check that the Docker service started successfully after installation.

sudo systemctl status docker

```
ubuntu@ubuntu:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-04-27 17:46:57 PKT; 45s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 6904 (dockerd)
      Tasks: 10
     Memory: 28.2M
        CPU: 439ms
     CGroup: /system.slice/docker.service
             └─6904 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.160463823+05:00" level=info msg="Restoring containers: start."
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.202281969+05:00" level=info msg="Deleting nftables IPv4 rules" erro>
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.229855908+05:00" level=info msg="Deleting nftables IPv6 rules" erro>
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.670995938+05:00" level=info msg="Loading containers: done."
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.685806017+05:00" level=info msg="Docker daemon" commit=6c91b92 cont>
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.685972195+05:00" level=info msg="Initializing buildkit"
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.716995373+05:00" level=info msg="Completed buildkit initialization"
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.724394259+05:00" level=info msg="Daemon has completed initializatio>
Apr 27 17:46:57 ubuntu dockerd[6904]: time="2026-04-27T17:46:57.724496316+05:00" level=info msg="API listen on /run/docker.sock"
Apr 27 17:46:57 ubuntu systemd[1]: Started Docker Application Container Engine.
lines 1-22/22 (END)
```

You should see Active: active (running) in the output. If it is not running, start it manually:

sudo systemctl start docker
sudo systemctl enable docker

1.8 Test Docker with a Quick Hello World

Run a basic container to confirm Docker is working correctly.

sudo docker run hello-world

```
ubuntu@ubuntu:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

ubuntu@ubuntu:~$
```

If Docker is installed correctly, it will download the hello-world image and print a confirmation message.

STEP 2

Give Wazuh Agent Permission to Access Docker

The Wazuh agent needs permission to read the Docker socket file (located at /var/run/docker.sock). Without this permission, the agent cannot see what Docker is doing. This is fixed by adding the wazuh user to the docker group.

2.1 Add the wazuh User to the docker Group

sudo usermod -aG docker wazuh

```
ubuntu@ubuntu:~$ sudo usermod -aG docker wazuh
ubuntu@ubuntu:~$ S
```

2.2 Verify the Group Was Added

We also need to check the Docker socket permission:

ls -la /var/run/docker.sock

```
ubuntu@ubuntu:~$ ls -la /var/run/docker.sock
srw-rw---- 1 root docker 0 Apr 27 17:46 /var/run/docker.sock
```

Check that the wazuh user is now a member of the docker group.

groups wazuh

```
ubuntu@ubuntu:~$ groups wazuh
wazuh : wazuh cowrie docker
ubuntu@ubuntu:~$
```

You should see docker listed in the output alongside other groups.

WHY THIS MATTERS

The Docker socket (/var/run/docker.sock) is a special file that allows programs to communicate with the Docker daemon. Only users in the docker group can access it. The Wazuh agent runs as the wazuh user, so it needs to be in this group to monitor container activity.

STEP 3

Install Python Docker SDK (Required by Wazuh)

The Wazuh Docker listener uses Python's Docker SDK to communicate with the Docker daemon. This package must be installed on the Ubuntu Server before the Docker listener will work.

3.1 Install pip (Python Package Manager)

sudo apt install -y python3-pip

```
ubuntu@ubuntu:~$ sudo apt install -y python3-pip
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (22.0.2+dfsg-1ubuntu0.7).
0 to upgrade, 0 to newly install, 0 to remove and 40 not to upgrade.
ubuntu@ubuntu:~$
```

3.2 Install the Docker SDK for Python

sudo PIP_BREAK_SYSTEM_PACKAGES=1 pip3 install docker==7.1.0 urllib3==1.26.20 requests==2.32.2

```
ubuntu@ubuntu:/$ sudo PIP_BREAK_SYSTEM_PACKAGES=1 pip3 install docker==7.1.0 urllib3==1.26.20 requests==2.32.2
Collecting docker==7.1.0
  Downloading docker-7.1.0-py3-none-any.whl (147 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 147.8/147.8 KB 1.5 MB/s eta 0:00:00
Collecting urllib3==1.26.20
  Downloading urllib3-1.26.20-py2.py3-none-any.whl (144 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 144.2/144.2 KB 16.3 MB/s eta 0:00:00
Collecting requests==2.32.2
  Downloading requests-2.32.2-py3-none-any.whl (63 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 63.9/63.9 KB 5.5 MB/s eta 0:00:00
Collecting charset-normalizer<4,>=2
  Downloading charset_normalizer-3.4.7-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_
64.whl (216 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 216.9/216.9 KB 4.8 MB/s eta 0:00:00
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests==2.32.2) (3.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/lib/python3/dist-packages (from requests==2.32.2) (2020
.6.20)
Installing collected packages: urllib3, charset-normalizer, requests, docker
  Attempting uninstall: urllib3
    Found existing installation: urllib3 1.26.5
    Not uninstalling urllib3 at /usr/lib/python3/dist-packages, outside environment /usr
    Can't uninstall 'urllib3'. No files were found to uninstall.
  Attempting uninstall: requests
    Found existing installation: requests 2.25.1
    Not uninstalling requests at /usr/lib/python3/dist-packages, outside environment /usr
    Can't uninstall 'requests'. No files were found to uninstall.
Successfully installed charset-normalizer-3.4.7 docker-7.1.0 requests-2.32.2 urllib3-1.26.20
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the syste
m package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
```

This installs the docker Python package that Wazuh's listener module uses internally to subscribe to Docker events.

NOTE
If you get a pip externally-managed-environment error on newer Ubuntu versions, use:

sudo pip3 install docker --break-system-packages
Or
sudo apt install python3-docker

Verify installation worked

python3 -c "import docker; print(docker.__version__)"

```
ubuntu@ubuntu:/$ python3 -c "import docker; print(docker.__version__)"
7.1.0
```

Test the DockerListener manually

sudo /var/ossec/wodles/docker/DockerListener

```
ubuntu@ubuntu:/$ sudo /var/ossec/wodles/docker/DockerListener
{"integration": "docker", "docker": {"Wodle event": "Started"}}
Docker service was started.
{"integration": "docker", "docker": {"Wodle event": "Connected to Docker service"}}
^C^CException ignored in: <module 'threading' from '/usr/lib/python3.10/threading.py'>
Traceback (most recent call last):
  File "/usr/lib/python3.10/threading.py", line 1567, in _shutdown
    lock.acquire()
KeyboardInterrupt:
```

STEP 4

Enable the Docker Listener in the Wazuh Agent Configuration

Now I tell the Wazuh agent to start its Docker monitoring module. This is done by editing the agent's main configuration file on the Ubuntu Server.

4.1 Open the Wazuh Agent Configuration File

sudo nano /var/ossec/etc/ossec.conf

```
ubuntu@ubuntu:~$ sudo nano /var/ossec/etc/ossec.conf
```

4.2 Find the <wodle> Section or Add at the End

Scroll through the file. Look for any existing <wodle> blocks. You need to add the Docker listener block. Place it just before the closing </ossec_config> tag:

```
<wodle name="docker-listener">

 <disabled>no</disabled>
 <interval>1m</interval>
 <attempts>5</attempts>
 <run_on_start>yes</run_on_start>
</wodle>
```

```
GNU nano 6.2                    /var/ossec/etc/ossec.conf *
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>

  <localfile>
    <log_format>json</log_format>
    <location>/var/log/test_chrome_history.log</location>
  </localfile>

  <wodle name="docker-listener">
   <disabled>no</disabled>
   <interval>1m</interval>
   <attempts>5</attempts>
   <run_on_start>yes</run_on_start>
  </wodle>


</ossec_config>
```

Configuration Field
What It Does
disabled: no
Enables the Docker listener module
interval: 1m
How often Wazuh polls Docker for events (10 minutes is the default)
attempts: 5
How many times Wazuh retries if it cannot connect to Docker
run_on_start: yes
Starts monitoring immediately when the Wazuh agent starts

4.3 Save and Close the File

Press CTRL+O then Enter to save. Then press CTRL+X to exit nano.

STEP 5

Restart the Wazuh Agent to Apply Changes

After making changes to the configuration file, the Wazuh agent must be restarted so it loads the new Docker listener settings.

5.1 Restart the Wazuh Agent

sudo systemctl restart wazuh-agent

5.2 Verify the Agent is Running

sudo systemctl status wazuh-agent

```
ubuntu@ubuntu:~$ sudo systemctl restart wazuh-agent
ubuntu@ubuntu:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-04-27 18:27:12 PKT; 5min ago
    Process: 10495 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 36 (limit: 6285)
     Memory: 827.0M
        CPU: 39.262s
     CGroup: /system.slice/wazuh-agent.service
             ├─10518 /var/ossec/bin/wazuh-execd
             ├─10526 /var/ossec/bin/wazuh-agentd
             ├─10540 /var/ossec/bin/wazuh-syscheckd
             ├─10550 /var/ossec/bin/wazuh-logcollector
             ├─10562 /var/ossec/bin/wazuh-modulesd
             └─10575 python3 wodles/docker/DockerListener

Apr 27 18:27:06 ubuntu systemd[1]: Starting Wazuh agent...
Apr 27 18:27:06 ubuntu env[10495]: Starting Wazuh v4.14.1...
Apr 27 18:27:07 ubuntu env[10495]: Started wazuh-execd...
Apr 27 18:27:08 ubuntu env[10495]: Started wazuh-agentd...
Apr 27 18:27:08 ubuntu env[10495]: Started wazuh-syscheckd...
Apr 27 18:27:09 ubuntu env[10495]: Started wazuh-logcollector...
Apr 27 18:27:10 ubuntu env[10495]: Started wazuh-modulesd...
Apr 27 18:27:12 ubuntu env[10495]: Completed.
Apr 27 18:27:12 ubuntu systemd[1]: Started Wazuh agent.
ubuntu@ubuntu:~$
```

Look for Active: active (running). If you see it, the agent has restarted successfully and the Docker listener is now active.

5.3 Add Rules: (On Manager Side)

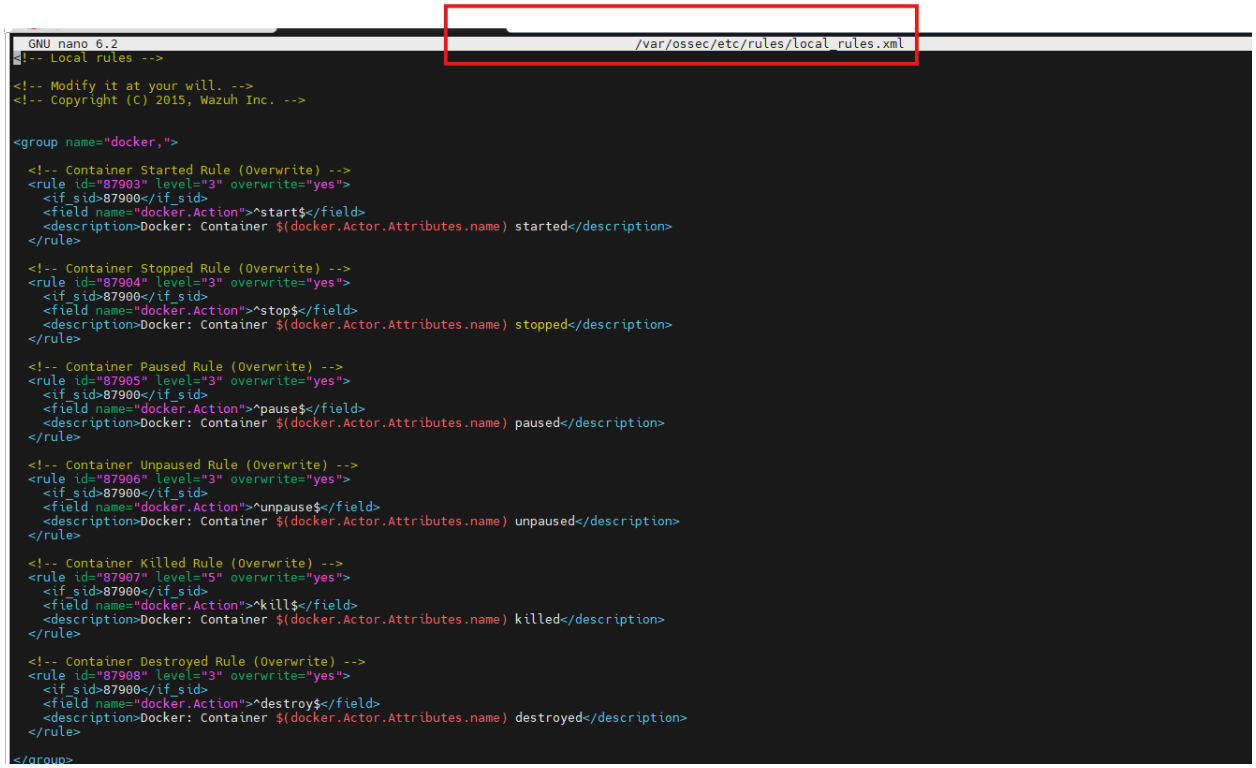
sudo nano /var/ossec/etc/rules/local_rules.xml

```
wazuh@fypserver:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
wazuh@fypserver:~$
```

```
<group name="docker,">

  <!-- Container Started Rule (Overwrite) -->

  <rule id="87903" level="3" overwrite="yes">

    <if_sid>87900</if_sid>

    <field name="docker.Action">^start$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name)
started</description>

  </rule>



  <!-- Container Stopped Rule (Overwrite) -->
```

```
  <rule id="87904" level="3" overwrite="yes">

    <if_sid>87900</if_sid>

    <field name="docker.Action">^stop$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name)
stopped</description>

  </rule>


  <!-- Container Paused Rule (Overwrite) -->

  <rule id="87905" level="3" overwrite="yes">

    <if_sid>87900</if_sid>

    <field name="docker.Action">^pause$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name)
paused</description>

  </rule>


  <!-- Container Unpaused Rule (Overwrite) -->

  <rule id="87906" level="3" overwrite="yes">

    <if_sid>87900</if_sid>

    <field name="docker.Action">^unpause$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name)
unpaused</description>

  </rule>


  <!-- Container Killed Rule (Overwrite) -->

  <rule id="87907" level="5" overwrite="yes">
```

```
    <if_sid>87900</if_sid>

    <field name="docker.Action">^kill$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name) killed</description>

  </rule>



  <!-- Container Destroyed Rule (Overwrite) -->

  <rule id="87908" level="3" overwrite="yes">

    <if_sid>87900</if_sid>

    <field name="docker.Action">^destroy$</field>

    <description>Docker: Container $(docker.Actor.Attributes.name)
destroyed</description>

  </rule>

</group>
```



Check the events on Manager side:

sudo grep "docker" /var/ossec/logs/alerts/alerts.json | tail -5


```
wazuh@fypserver:~$ sudo grep "docker" /var/ossec/logs/alerts/alerts.json | tail -5
[sudo] password for wazuh:
{"timestamp":"2026-04-27T21:58:21.831+0500","rule":{"level":4,"description":"Docker: Network bridge disconnected","id":"87929","firedtimes":2,"mail":false,"groups":["docker"]},"agent":{"id":"002","name":"Ubuntu-Agent","ip":"10.10.80.68"},"manager":{"name":"fypserver"},"id":"1777309101.18226614","decoder":{"name":"json"},"data":{"integration":"docker","docker":{"Type":"network","Action":"disconnect","Actor":{"ID":"1952318b7eddddd1732002c2ee9f40d1a0203aefc9d3b1b6071edffbfdac2f976","Attributes":{"container":"0fe471851167b41cfba68e54df8ebd8fadac38ae92842f39e31c0268694fde12","name":"bridge","type":"bridge"}},"scope":"local","time":"1777309101","timeNano":"1777309101833049600.000000"}},"location":"Wazuh-Docker"}
{"timestamp":"2026-04-27T21:58:21.853+0500","rule":{"level":3,"description":"Docker: Container festive_wilson stopped","id":"87904","firedtimes":1,"mail":false,"groups":["docker"]},"agent":{"id":"002","name":"Ubuntu-Agent","ip":"10.10.80.68"},"manager":{"name":"fypserver"},"id":"1777309101.18227553","full_log":"{\"integration\": \"docker\", \"docker\": {\"Type\": \"container\", \"Action\": \"stop\", \"Actor\": {\"ID\": \"0fe471851167b41cfba68e54df8ebd8fadac38ae92842f39e31c0268694fde12\", \"Attributes\": {\"image\": \"nginx\", \"maintainer\": \"NGINX Docker Maintainers <docker-maint@nginx.com>\", \"name\": \"festive_wilson\"}}, \"scope\": \"local\", \"time\": 1777309101, \"timeNano\": 1777309101855706051}}","decoder":{"name":"json"},"data":{"integration":"docker","docker":{"Type":"container","Action":"stop","Actor":{"ID":"0fe471851167b41cfba68e54df8ebd8fadac38ae92842f39e31c0268694fde12","Attributes":{"image":"nginx","maintainer":"NGINX Docker Maintainers <docker-maint@nginx.com>","name":"festive_wilson"}},"scope":"local","time":"1777309101","timeNano":"1777309101855706112.000000"}},"location":"Wazuh-Docker"}
{"timestamp":"2026-04-27T22:14:31.030+0500","rule":{"level":5,"description":"Docker: Container lucid_beaver killed","id":"87907","firedtimes":1,"mail":false,"groups":["docker"]},"agent":{"id":"002","name":"Ubuntu-Agent","ip":"10.10.80.68"},"manager":{"name":"fypserver"},"id":"1777310071.19574131","full_log":"{\"integration\": \"docker\", \"docker\": {\"Type\": \"container\", \"Action\": \"kill\", \"Actor\": {\"ID\": \"248d3cf8a9b79fb8a8f1ea040c11e6d818c7a88aaa19c3844a158757823333a2\", \"Attributes\": {\"image\": \"nginx\", \"maintainer\": \"NGINX Docker Maintainers <docker-maint@nginx.com>\", \"name\": \"lucid_beaver\", \"signal\": \"3\"}}, \"scope\": \"local\", \"time\": 1777310071, \"timeNano\": 1777310071003318974}}","decoder":{"name":"json"},"data":{"integration":"docker","docker":{"Type":"container","Action":"kill","Actor":{"ID":"248d3cf8a9b79fb8a8f1ea040c11e6d818c7a88aaa19c3844a158757823333a2","Attributes":{"image":"nginx","maintainer":"NGINX Docker Maintainers <docker-maint@nginx.com>","name":"lucid_beaver","signal":"3"}},"scope":"local","time":"1777310071","timeNano":"1777310071003319040.000000"}},"location":"Wazuh-Docker"}
{"timestamp":"2026-04-27T22:14:31.167+0500","rule":{"level":4,"description":"Docker: Network bridge disconnected","id":"87929","firedtimes":1,"mail":false,"groups":["docker"]},"agent":{"id":"002","name":"Ubuntu-Agent","ip":"10.10.80.68"},"manager":{"name":"fypserver"},"id":"1777310071.19575097","decoder":{"name":"json"},"data":{"integration":"docker","docker":{"Type":"network","Action":"disconnect","Actor":{"ID":"1952318b7eddddd1732002c2ee9f40d1a0203aefc9d3b1b6071edffbfdac2f976","Attributes":{"container":"248d3cf8a9b79fb8a8f1ea040c11e6d818c7a88aaa19c3844a158757823333a2","name":"bridge","type":"bridge"}},"scope":"local","time":"1777310071","timeNano":"1777310071138370816.000000"}},"location":"Wazuh-Docker"}
{"timestamp":"2026-04-27T22:14:31.186+0500","rule":{"level":3,"description":"Docker: Container lucid_beaver stopped","id":"87904","firedtimes":1,"mail":false,"groups":["docker"]},"agent":{"id":"002","name":"Ubuntu-Agent","ip":"10.10.80.68"},"manager":{"name":"fypserver"},"id":"1777310071.19576036","full_log":"{\"integration\": \"docker\", \"docker\": {\"Type\": \"container\", \"Action\": \"stop\", \"Actor\": {\"ID\": \"248d3cf8a9b79fb8a8f1ea040c11e6d818c7a88aaa19c3844a158757823333a2\", \"Attributes\": {\"image\": \"nginx\", \"maintainer\": \"NGINX Docker Maintainers <docker-maint@nginx.com>\", \"name\": \"lucid_beaver\"}}, \"scope\": \"local\", \"time\": 1777310071, \"timeNano\": 1777310071159706922}}","decoder":{"name":"json"},"data":{"integration":"docker","docker":{"Type":"container","Action":"stop","Actor":{"ID":"248d3cf8a9b79fb8a8f1ea040c11e6d818c7a88aaa19c3844a158757823333a2","Attributes":{"image":"nginx","maintainer":"NGINX Docker Maintainers <docker-maint@nginx.com>","name":"lucid_beaver"}},"scope":"local","time":"1777310071","timeNano":"1777310071159706880.000000"}},"location":"Wazuh-Docker"}
```


STEP 6

Run Docker Containers to Generate Events

Now I generate real Docker events so that Wazuh has something to detect and log. This is done by running, stopping, and interacting with containers on the Ubuntu Server.

6.1 Pull a Docker Image

Pull the official nginx web server image from Docker Hub. This action generates an image pull event.

sudo docker pull nginx

```
ubuntu@ubuntu:~$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
ubuntu@ubuntu:~$
```

6.2 Start a Container

Start an nginx container in detached mode (running in the background). This generates a container start event.

sudo docker run -d --name test-nginx -p 8080:80 nginx

```
ubuntu@ubuntu:~$ sudo docker run -d --name test-nginx -p 8080:80 nginx
25abcbee8f1c16319d3d72d1b4cb363a4efa10e4faec0ca8c47edeeaafb86a5c
ubuntu@ubuntu:~$
```

Here, -d means detached (background), --name gives the container a name, and -p 8080:80 maps port 8080 on the host to port 80 in the container.

6.3 List Running Containers

Verify that the container is actually running.

sudo docker ps

```
ubuntu@ubuntu:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
25abcbee8f1c   nginx     "/docker-entrypoint.…"   20 seconds ago   Up 19 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   test-nginx
ubuntu@ubuntu:~$
```

You should see test-nginx listed with a status of Up.

6.4 Stop the Container

Stopping the container generates a stop event that Wazuh will capture.

sudo docker stop test-nginx

```
ubuntu@ubuntu:~$ sudo docker stop test-nginx
test-nginx
ubuntu@ubuntu:~$
```

6.5 Remove the Container

Remove the stopped container. This generates a destroy event.

sudo docker rm test-nginx

```
ubuntu@ubuntu:~$ sudo docker rm test-nginx
test-nginx
ubuntu@ubuntu:~$
```

6.6 Pull Another Image for Variety

Pull a second image to generate more events for testing.

sudo docker pull alpine
sudo docker run -it --rm alpine echo 'Hello from Alpine'

```
ubuntu@ubuntu:~$ sudo docker pull alpine
sudo docker run -it --rm alpine echo 'Hello from Alpine'
Using default tag: latest
latest: Pulling from library/alpine
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Image is up to date for alpine:latest
docker.io/library/alpine:latest
Hello from Alpine
ubuntu@ubuntu:~$
```

WHAT WAZUH CAPTURES

Every Docker action above pull, create, start, stop, destroy is captured as a separate event by the Wazuh Docker listener and forwarded to the Wazuh Manager automatically.

STEP 7

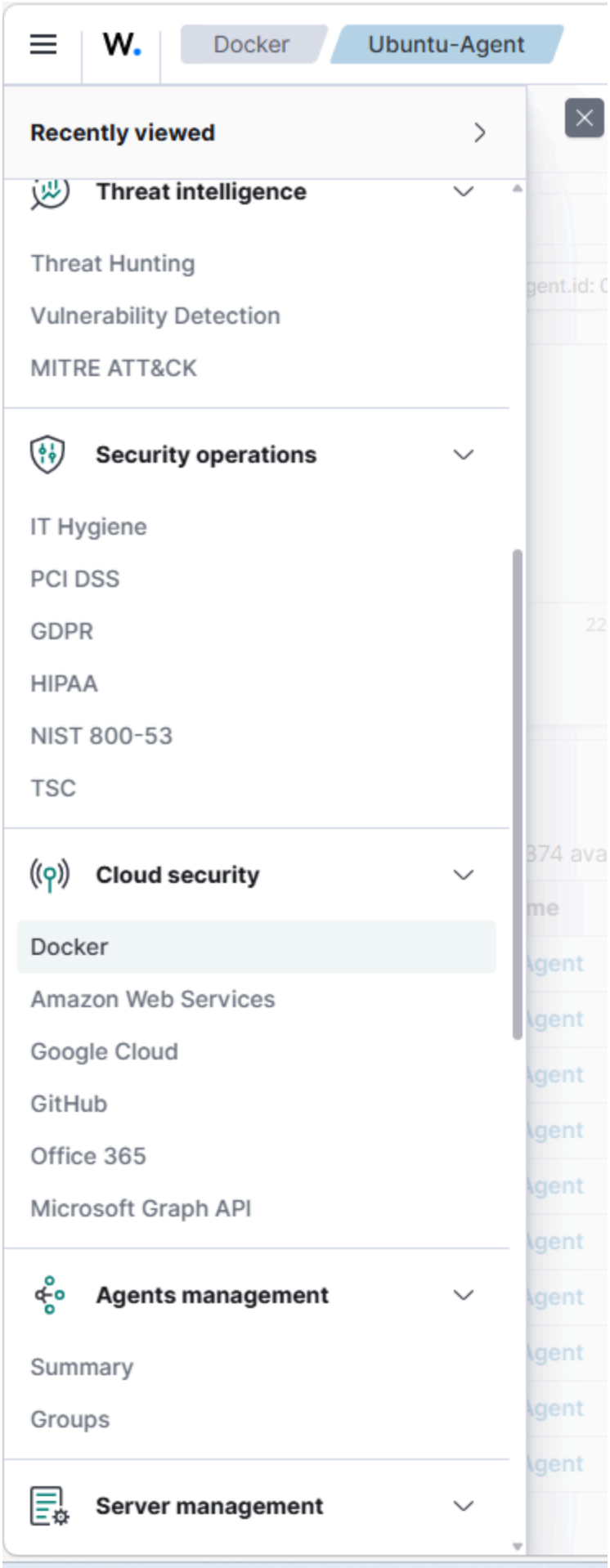
Verify Detection on Wazuh Dashboard

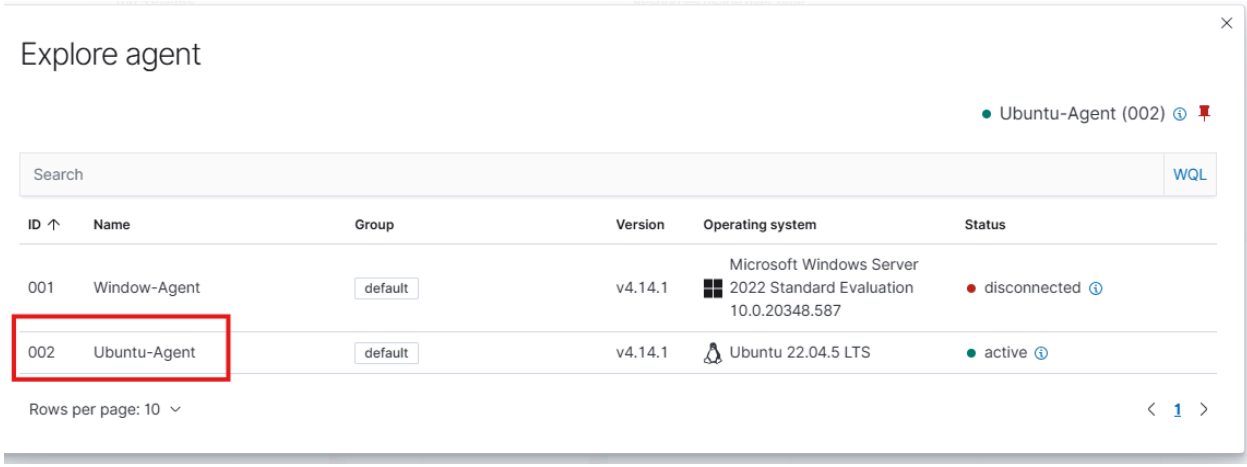
Log in to the Wazuh Dashboard from your Ubuntu Desktop machine (where the Wazuh Manager is installed). Open your browser and go to:

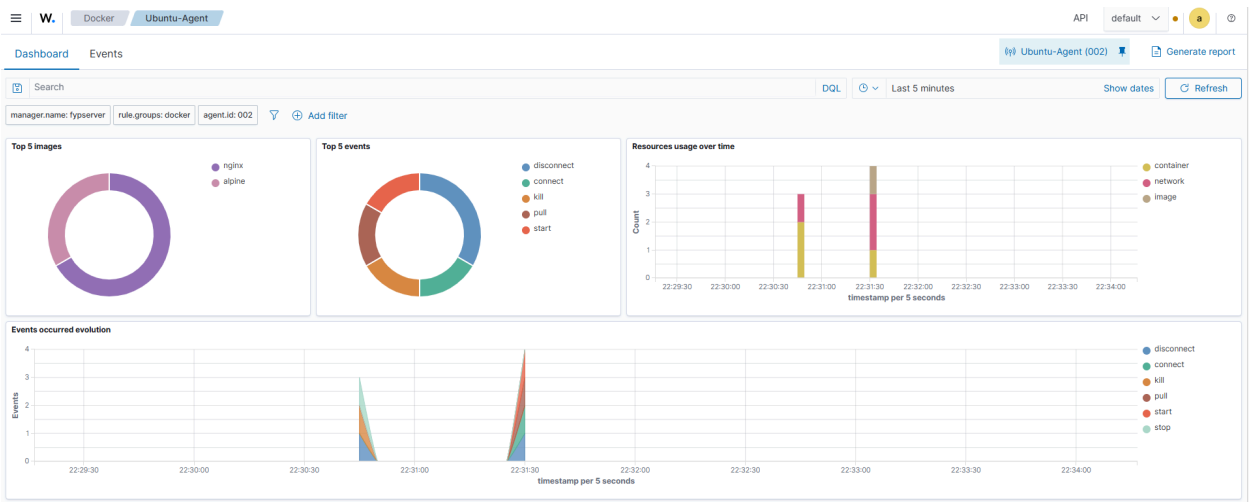
https://<wazuh-manager-ip>:443

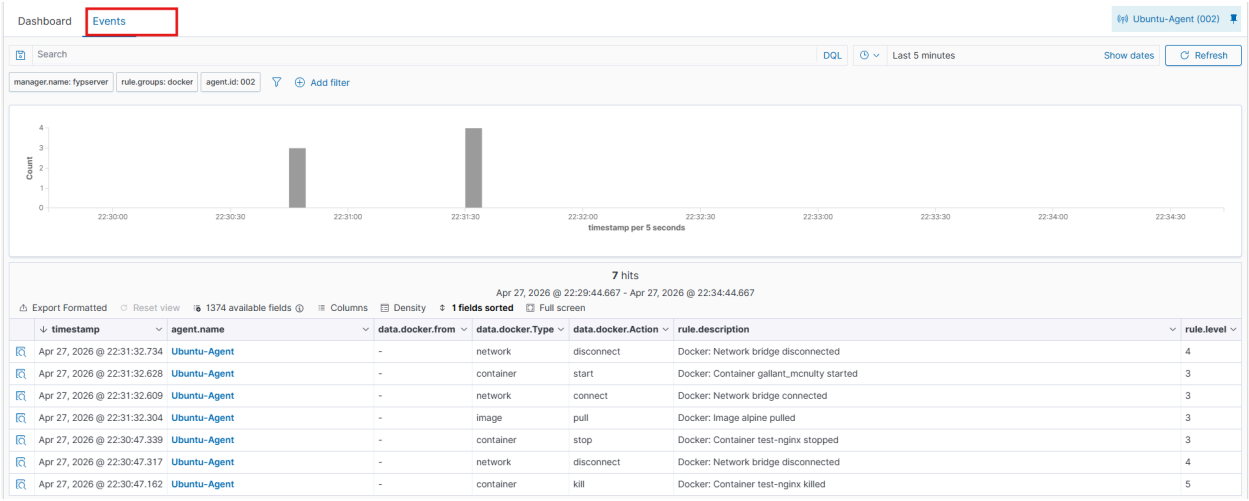
7.1 Navigate to the Docker Listener Module

- In the left sidebar, go to Modules.
- Click on Docker listener.
- Select your Ubuntu Server agent from the agent list.
- You will see a summary of all captured Docker events including images pulled, containers started, and containers stopped.

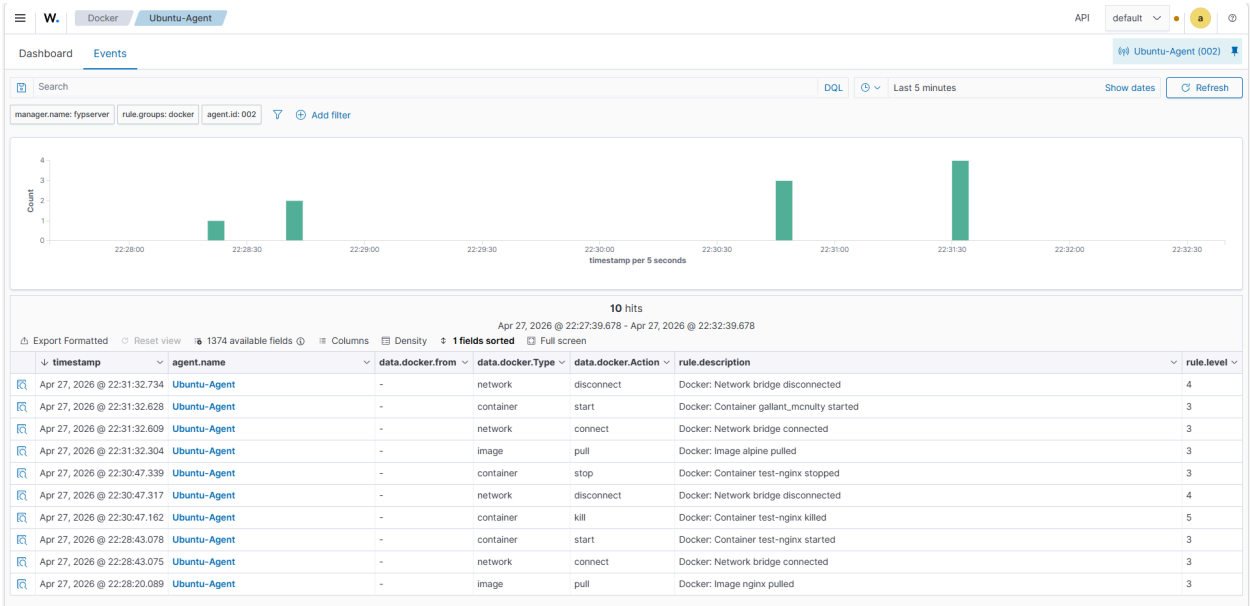

W.
Docker
Ubuntu-Agent
Recently viewed
Threat intelligence
Threat Hunting
Vulnerability Detection
MITRE ATT&CK
Security operations
IT Hygiene
PCI DSS
GDPR
HIPAA
NIST 800-53
TSC
Cloud security
Docker
Amazon Web Services
Google Cloud
GitHub
Office 365
Microsoft Graph API
Agents management
Summary
Groups
Server management





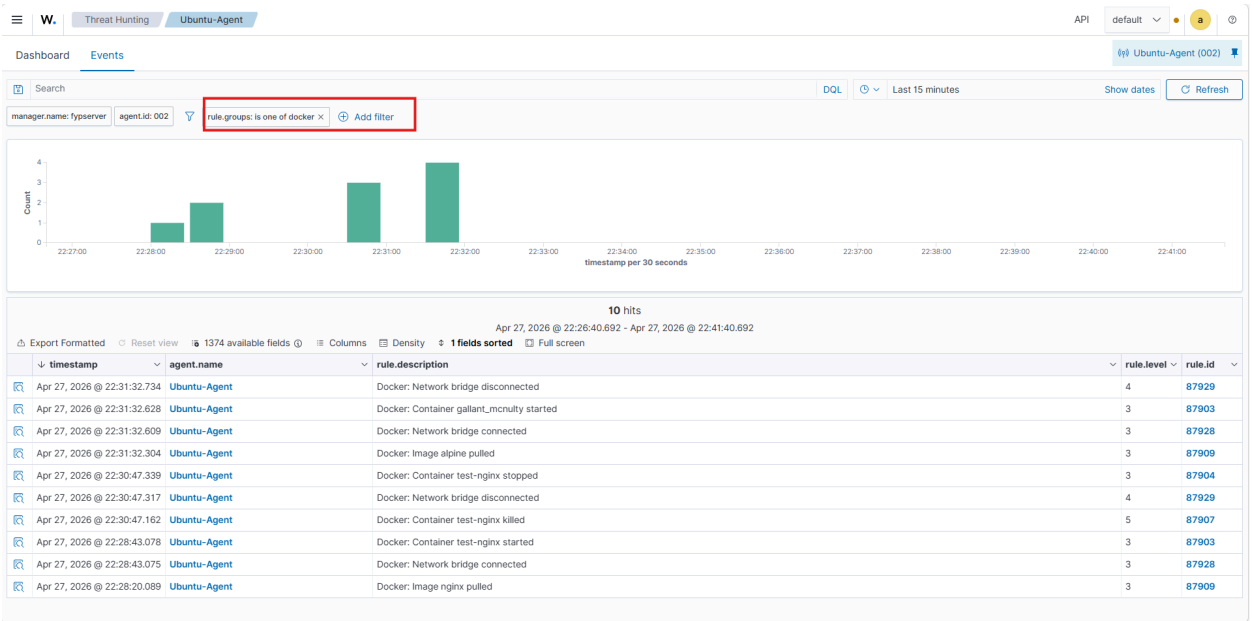


timestamp	agent.name	data.docker.from	data.docker.Type	data.docker.Action	rule.description	rule.level
Apr 27, 2026 @ 22:31:32.734	Ubuntu-Agent	-	network	disconnect	Docker: Network bridge disconnected	4
Apr 27, 2026 @ 22:31:32.628	Ubuntu-Agent	-	container	start	Docker: Container gallant_mcnulty started	3
Apr 27, 2026 @ 22:31:32.609	Ubuntu-Agent	-	network	connect	Docker: Network bridge connected	3
Apr 27, 2026 @ 22:31:32.304	Ubuntu-Agent	-	image	pull	Docker: Image alpine pulled	3
Apr 27, 2026 @ 22:30:47.339	Ubuntu-Agent	-	container	stop	Docker: Container test-nginx stopped	3
Apr 27, 2026 @ 22:30:47.317	Ubuntu-Agent	-	network	disconnect	Docker: Network bridge disconnected	4
Apr 27, 2026 @ 22:30:47.162	Ubuntu-Agent	-	container	kill	Docker: Container test-nginx killed	5



7.2 Check the Events Tab

- Go to your agent page and click the Events tab.
- Filter by rule.groups: docker to show only Docker-related alerts.
- You should see events for each action you performed: image pull, container start, container stop, container destroy.



Details Alerts

Table JSON

@timestamp	Apr 27, 2026 @ 22:31:32.628
_index	wazuh-alerts-4.x-2026.04.27
agent.id	002
agent.ip	10.10.80.68
agent.name	Ubuntu-Agent
data.docker.Action	start
data.docker.Actor.Attributes.image	alpine
data.docker.Actor.Attributes.name	gallant_mcnulty
data.docker.Actor.ID	2c3fc56038f4e0ff0f98e92b510021ed3bb49afaad274035e308162db9ca8410
data.docker.Type	container
data.docker.scope	local
data.docker.time	1777311092
data.docker.timeNano	1777311092619109120.000000
data.integration	docker
decoder.name	json
full_log	{"integration": "docker", "docker": {"Type": "container", "Action": "start", "Actor": {"ID": "2c3fc56038f4e0ff0f98e92b510021ed3bb49afaad274035e308162db9ca8410", "Attributes": {"image": "alpine", "name": "gallant_mcnulty"}}, "scope": "local", "time": 1777311092, "timeNano": 1777311092619109184}}
id	1777311092.21270406
input.type	log
location	Wazuh-Docker
manager.name	fypserver
rule.description	Docker: Container gallant_mcnulty started
rule.firedtimes	2
rule.groups	docker
rule.id	87903
rule.level	3

Summary:

In this task, I implemented Docker container monitoring using Wazuh SIEM in a controlled lab environment. The main objective was to capture real-time Docker container events such as container start, stop, kill, destroy, and image pull operations, and generate security alerts on the Wazuh Dashboard without needing someone to check container logs manually.

First, I installed Docker Engine on my Ubuntu Server where the Wazuh Agent is already installed. I removed old Docker packages, added Docker's official GPG key and repository, then installed Docker Engine using apt. I verified Docker was running with the hello-world container.

The Wazuh agent needed permission to read the Docker socket file at `/var/run/docker.sock`. I added the wazuh user to the docker group using `usermod -aG docker wazuh` so the agent could communicate with the Docker daemon.

I installed the Python Docker SDK which is required by the Wazuh Docker listener. I installed docker version 7.1.0, urllib3 version 1.26.20, and requested version 2.32.2 using pip3. The Wazuh Docker listener uses this SDK to subscribe to Docker events.

On the Wazuh Agent, I edited /var/ossec/etc/ossec.conf and added the docker-listener wodle block with disabled set to no, interval set to 1m, attempts set to 5, and run_on_start set to yes. This enables the listener to start monitoring immediately when the agent starts.

On the Wazuh Manager side on my Ubuntu Desktop, I created custom rules in /var/ossec/etc/rules/local_rules.xml. I identified that the default Wazuh rules were looking for docker.status field but the Docker listener was sending docker.Action field. This field mismatch was preventing container alerts from appearing. I created overwrite rules for containers started (rule 87903), stopped (87904), paused (87905), unpaused (87906), killed (87907), and destroyed (87908) using the correct docker.Action field.

After configuration, I restarted both the Wazuh agent and manager. To test the system, I pulled the nginx image, started a container with docker run -d --name test-nginx nginx, stopped it with docker stop test-nginx, and removed it with docker rm test-nginx. I also pulled the alpine image for variety.

To verify, I logged into the Wazuh Dashboard and navigated to Modules then Docker listener. I selected my Ubuntu Agent and saw all captured Docker events including images pulled, containers started, and containers stopped. I filtered the Events tab by rule.groups: docker which displayed all Docker-related alerts successfully. The dashboard showed multiple hits confirming the Docker listener was working properly.

This implementation shows how Wazuh SIEM can monitor Docker container activities in real-time without expensive commercial tools. It captures all container lifecycle events and generates alerts on a centralized dashboard. It is good for lab-based testing and for training SOC teams to detect suspicious container activities and potential container-based attacks.